OUTLIER DETECTION

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_excel("tips.xlsx")
def detect_outliers_total_bill(df, column_name="total_bill", z_threshold=2):
    data_values = df[column_name].dropna().values
    mean = np.mean(data_values)
    stddev = np.std(data_values)
    z_scores = (data_values - mean) / stddev
    z_outliers = np.where(np.abs(z_scores) > z_threshold)[0]
    q1 = np.percentile(data_values, 25)
    q3 = np.percentile(data_values, 75)
    iqr = q3 - q1
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr
    iqr_outliers = np.where((data_values < lower_bound) | (data_values > upper_bound))[0]
    print(f"\n==== {column_name.upper()} ====")
    print(f"Mean: {mean:.2f}")
    print(f"Standard Deviation: {stddev:.2f}")
    print(f"IQR: {iqr:.2f} | Lower Bound: {lower_bound:.2f} | Upper Bound: {upper_bound:.2f}")
    print(f"Outliers using Z-Score: {data_values[z_outliers]}")
    print(f"Outliers using IQR: {data_values[iqr_outliers]}")
    plt.figure(figsize=(6, 5))
    sns.boxplot(y=data_values, color="skyblue")
    plt.title(f"Boxplot of {column_name}")
    plt.ylabel(column_name)
    plt.grid(True)
    plt.show()
detect_outliers_total_bill(df, "total_bill")
```

07 / 07 / 2025

EXPERIMENT NO: 8

OUTLIER DETECTION USING BOXPLOT, IQR, Z-SCORE

AIM

To implement python program for data preprocessing operations -outliers and/or inconsistent management data value using Boxplot, IQR, and Z-score.

ALGORITHM

Step 1 : Import necessary libraries
Step 2 : Load the dataset.
Step 3 : Define a function detect_outliers to handle outliers detection for specific column and label.
Step 4 : Extract the column_values from the dataset using data[column_name] values.
Step 5 : Calculate the mean and standard deviation using numpy.mean() and numpy.std(), then compute z_score for all values.
Step 6 : Set a z_score threshold (z) for detecting outliers
Step 7 : Using np.percentile() and drew the interquantile range (IQR). Calculate the first and third quartiles
Step 8 : Use IQR to calculate the lower and upper bounds for outlier detection (Q1 - 1.5 * IQR and Q3 + 1.5 * IQR)
Step 9 : Identify the outliers by comparing z-score and IQR

OUTPUT

==== TOTAL_BILL ====

Mean: 19.79

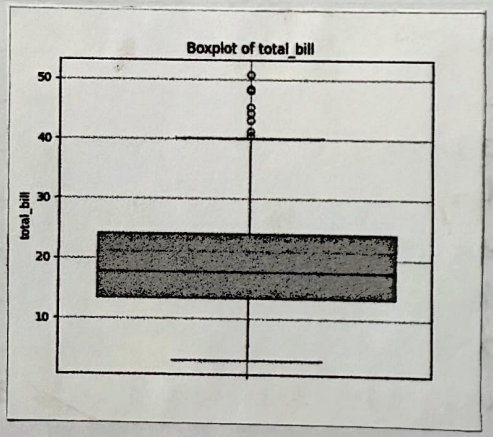
Standard Deviation: 8.88

IQR: 10.78 | Lower Bound: -2.82 | Upper Bound: 40.30

Outliers using Z-Score: [39.42 38.01 48.27 40.17 44.3 38.07 41.19 48.17 50.81 45.35 40.55 43.11

38.73 48.33]

Outliers using IQR: [48.27 44.3 41.19 48.17 50.81 45.35 40.55 43.11 48.33]



/ /

methods.

Step 10 : Plot the boxplot for visualizing the distribution using plt.boxplot() and plt.show()

Step 11 : Call the function with the specific column and label for analysis.

Step 12 : STOP

RESULT

The program was successfully executed and output was obtained.

11/09/25

FEEDFORWARD NN

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
import numpy as np
import matplotlib.pyplot as plt
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
def create_model(hidden_units=[512, 256, 128]):
    model = models.Sequential([
        layers.Flatten(input_shape=(32, 32, 3)),
        layers.Dense(hidden_units[0], activation='relu'),
        layers.Dense(hidden_units[1], activation='relu'),
        layers.Dense(hidden_units[2], activation='relu'),
        layers.Dense(10, activation='softmax')
    ])
    return model

model = create_model()
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
history = model.fit(x_train, y_train_cat, epochs=5, batch_size=64, validation_split=0.2, verbose=1)
test_loss, test_acc = model.evaluate(x_test, y_test_cat)
print(f"\nTest Accuracy: {round(test_acc * 100, 2)}%")
plt.figure(figsize=(8, 5))
```

07 / 07 / 25 | 6

EXPERIMENT NO : 3

CIFAR-10 CLASSIFICATION

AIM

To implement a python program for the classification of images in the CIFAR-10 dataset using neural network.

ALGORITHM

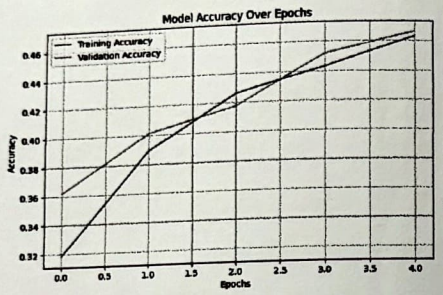
Step 1 : Import required libraries
Step 2 : Load CIFAR-10 dataset and split into training and testing sets.
Step 3 : Normalize image pixel values to range [0, 1]
Step 4 : Convert class labels into one-hot encoded vectors
Step 5 : Define a sequential model :
- Flatten layer (32, 32, 3)
- Dense (512, ReLU)
- Dense (256, ReLU)
- Dense (128, ReLU)
- Dense (10, softmax)

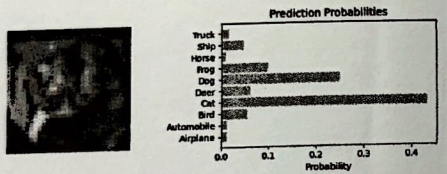
Step 6 : Compile the model with Adam optimizer, categorical-cross-entropy loss and accuracy metric
Step 7 : Train the model with training data for 5 epochs; batch size = 64, with 20 % validation split.
Step 8 : Evaluate model on test data and record test accuracy.

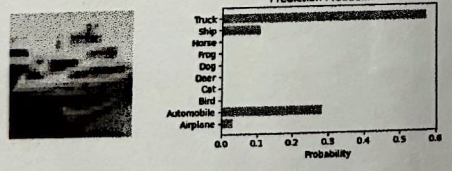
```
plt.plot(history.history['accuracy'],
label='Training Accuracy')
plt.plot(history.history['val_accuracy'],
label='Validation Accuracy')
plt.title("Model Accuracy Over Epochs")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()
class_labels = ["Airplane", "Automobile",
"Bird", "Cat", "Deer", "Dog", "Frog",
"Horse", "Ship", "Truck"]
num_images = 3
sample_images = x_test[:num_images]
predictions =
model.predict(sample_images)
def plot_probability_meter(predictions,
image):
  plt.figure(figsize=(8, 3))
  plt.subplot(1, 2, 1)
  plt.imshow(image)
  plt.axis('off')
  plt.subplot(1, 2, 2)
  plt.barh(class_labels, predictions,
color='skyblue')
  plt.xlabel("Probability")
  plt.title("Prediction Probabilities")
  plt.tight_layout()
  plt.show()
for i in range(num_images):
  plot_probability_meter(predictions[i],
sample_images[i])
```

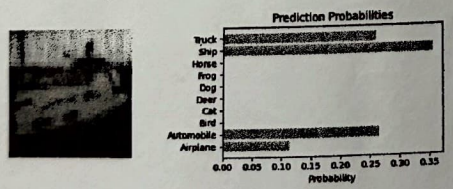
OUTPUT

Test Accuracy: 47.16%









step 9 : plot training vs validation accuracy over epochs.
step 10 : Predict on a few test samples and display probability distribution with bar chart.
step 11 : Predict on a few test samples and display probability distribution with bar chart.
step 12 : Analyze result and conclude.
step 13 : STOP

RESULT

The program was successfully executed and output was obtained.

KAIMING AND XAVIER

```
import tensorflow as tf

from tensorflow.keras import layers, models, initializers, regularizers

from tensorflow.keras.datasets import cifar10

from tensorflow.keras.utils import to_categorical

import matplotlib.pyplot as pl

(x_train, y_train), (x_test, y_test) = cifar10.load_data()

x_train, x_test = x_train / 255.0, x_test / 255.0

y_train_cat = to_categorical(y_train, 10)

y_test_cat = to_categorical(y_test, 10)

def create_model(initializer, dropout_rate=0.3, kernel_regularizer=regularizers.l2(0.01)):

    model = models.Sequential([

        layers.Flatten(input_shape=(32, 32, 3)),

        layers.Dense(512, activation='relu', kernel_initializer=initializer, kernel_regularizer=kernel_regularizer),

        layers.Dropout(dropout_rate),

        layers.Dense(256, activation='relu', kernel_initializer=initializer, kernel_regularizer=kernel_regularizer),

        layers.Dropout(dropout_rate),

        layers.Dense(128, activation='relu', kernel_initializer=initializer, kernel_regularizer=kernel_regularizer),

        layers.Dropout(dropout_rate),

        layers.Dense(64, activation='relu', kernel_initializer=initializer, kernel_regularizer=kernel_regularizer),

        layers.Dropout(dropout_rate),

        layers.Dense(32, activation='relu', kernel_initializer=initializer, kernel_regularizer=kernel_regularizer),

        layers.Dropout(dropout_rate),

        layers.Dense(10, activation='softmax')

    ])

    return model

xavier_initializer = initializers.glorot_normal()

kaiming_initializer = initializers.he_normal()

xavier_model = create_model(xavier_initializer, dropout_rate=0.3)

kaiming_model = create_model(kaiming_initializer, dropout_rate=0.3)

xavier_model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

kaiming_model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

xavier_history = xavier_model.fit(x_train, y_train_cat, epochs=10, batch_size=64, validation_split=0.2, verbose=1)

kaiming_history = kaiming_model.fit(x_train, y_train_cat,
```


EXPERIMENT NO: 4

KAIMING AND XAVIER INITIALIZATION

AIM

To write a python program to compare the impact of Kaiming and Xavier weight initialization and dropout on a neural network's performance during training and validation using the CIFAR-10 dataset.

ALGORITHM

Step 1 : Import required libraries.

Step 2 : Load CIFAR-10 dataset and split into training and testing sets.

Step 3 : Normalize image pixel values to range [0, 1].

Step 4 : Convert class labels into one-hot-encoded vectors.

Step 5 : Define a function to create a Sequential model with

- Flatten Layer (32, 32, 3)
- Dense (256, ReLU, initializer, ReLU, L2)
- Dense (128, ReLU, initializer, ReLU, L2)
- Dense (64, ReLU, initializer, ReLU, L2)
- Dense (32, ReLU, initializer, ReLU, L2)
- Dense (10, Softmax)

Step 6 : Create two models using different weight initializers :

- Xavier initializer (Glorot Normal)

```
epochs=10, batch_size=64,
validation_split=0.2, verbose=1)

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)

plt.plot(xavier_history.history['accuracy'],
label='Xavier Train')

plt.plot(xavier_history.history['val_accurac
y'], label='Xavier Val')

plt.title('Xavier Initialization')

plt.xlabel('Epochs')

plt.ylabel('Accuracy')

plt.legend()

plt.subplot(1, 2, 2)

plt.plot(kaiming_history.history['accuracy']
, label='Kaiming Train')

plt.plot(kaiming_history.history['val_accur
acy'], label='Kaiming Val')

plt.title('Kaiming Initialization')

plt.xlabel('Epochs')

plt.ylabel('Accuracy')

plt.legend()

plt.tight_layout()

plt.show()

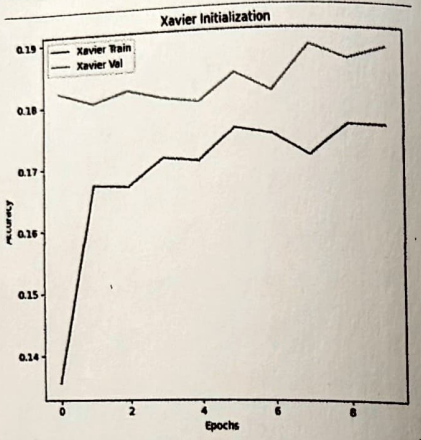
test_loss_x, test_acc_x =
xavier_model.evaluate(x_test, y_test_cat)

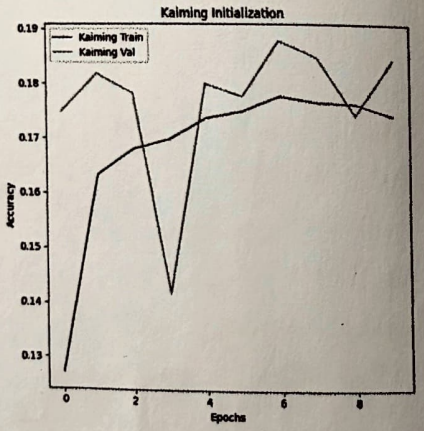
test_loss_k, test_acc_k =
kaiming_model.evaluate(x_test,
y_test_cat)


print(f"\nXavier Test Accuracy:
{round(test_acc_x * 100, 2)}%")

print(f"Kaiming Test Accuracy:
{round(test_acc_k * 100, 2)}%")
```

OUTPUT





Kaiming initializer (He Normal).

Step 7 : Compile both models with Adam optimizer, categorical cross entropy - loss, and accuracy metric.

Step 8 : Train both models for 10 epochs, batch size = 64, with 20% validation split and store history.

Step 9 : Plot training vs validation accuracy for both Xavier and Kaiming initialization.

Step 10: Evaluate both models on test data and record test accuracy.

Step 11 : Compare results of Xavier vs Kaiming initialization.

Step 12 : Analyze results and conclude which initialization performs better.

RESULT

The program was successfully executed and output was obtained.

GRADIENT DESCENT

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras import layers, models, optimizers
import matplotlib.pyplot as plt
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
train_images, test_images = train_images / 255.0, test_images / 255.0
train_images = train_images.reshape(train_images.shape[0], 28, 28, 1)
test_images = test_images.reshape(test_images.shape[0], 28, 28, 1)
train_labels = to_categorical(train_labels)
test_labels = to_categorical(test_labels)

def create_model():
    model = models.Sequential([
        layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
        layers.MaxPooling2D((2, 2)),
        layers.Conv2D(64, (3, 3), activation='relu'),
        layers.MaxPooling2D((2, 2)),
        layers.Conv2D(64, (3, 3), activation='relu'),
        layers.Flatten(),
        layers.Dense(64, activation='relu'),
        layers.Dense(10, activation='softmax')
    ])
    return model

optimizers_dict = {
    "SGD": optimizers.SGD(learning_rate=0.01),
    "SGD+Momentum": optimizers.SGD(learning_rate=0.01, momentum=0.9),
    "SGD+Nesterov": optimizers.SGD(learning_rate=0.01, momentum=0.9, nesterov=True),
    "Adam": optimizers.Adam(learning_rate=0.001),
    "RMSprop": optimizers.RMSprop(learning_rate=0.001),
    "Adagrad": optimizers.Adagrad(learning_rate=0.01)
}

histories = {}

for name, optimizer in optimizers_dict.items():
    print(f"\nTraining with optimizer: {name}")
    model = create_model()
    model.compile(optimizer=optimizer, loss="categorical_crossentropy", metrics=["accuracy"])
```

28 / 07 / 2025

EXPERIMENT NO:5

GRADIENT DESCENT VARIANTS AND ADAPTIVE OPTIMIZATION ALGORITHMS

AIM

To implement a python program to show the difference in performance when using Gradient Descent, Stochastic GD, GD with momentum, GD with Nesterov moment along with different adaptive optimization algorithms.

ALGORITHM

Step 1 : Import necessary libraries
Step 2 : Load MNIST dataset and split into training and testing sets.
Step 3 : Normalize image pixel values to range [0,1] / (255.0)
Step 4 : Reshape images to include channel dimensions.
Step 5 : Convert class labels into one-hot encoded vectors
Step 6 : Define a create_model() function returning a Sequential CNN.
Step 7 : Define the optimizer dictionary to compare Stochastic Gradient Descent, SGD with Momentum, SGD + Nesterov, Adam, RMSProp, Adagrad.
Step 8 : For each optimizer
Step 8.1 : Instantiate a fresh model via create_model()

```
    history = model.fit(train_images,
train_labels, epochs=5, batch_size=64,
validation_split=0.1, verbose=1)

    histories[name] = history

plt.figure(figsize=(12, 6))

for name, history in histories.items():

    plt.plot(history.history['val_accuracy'],
label=f"{name} Val Acc")

plt.title("Validation Accuracy
Comparison")

plt.xlabel("Epochs")

plt.ylabel("Accuracy")

plt.legend()

plt.grid(True)

plt.show()


plt.figure(figsize=(12, 6))

for name, history in histories.items():

    plt.plot(history.history['val_loss'],
label=f"{name} Val Loss")

plt.title("Validation Loss Comparison")

plt.xlabel("Epochs")

plt.ylabel("Loss")

plt.legend()

plt.grid(True)

plt.show()


for name, optimizer in
optimizers_dict.items():

    model = create_model()

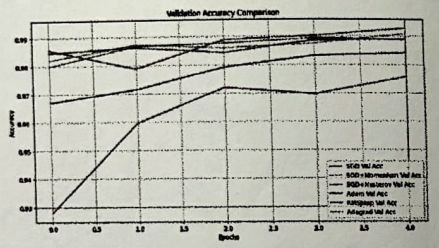
    model.compile(optimizer=optimizer,
loss="categorical_crossentropy",
metrics=["accuracy"])

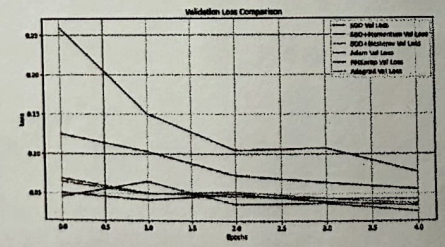
    model.fit(train_images, train_labels,
epochs=5, batch_size=64, verbose=0)

    test_loss, test_acc =
model.evaluate(test_images, test_labels,
verbose=0)

    print(f"{name} Test Accuracy:
{round(test_acc * 100, 2)}%")
```

OUTPUT





/ /

11

Step 8.2: compile with that optimizer, loss='categorical_crossentropy", metrics=['accuracy']

Step 8.3: Train with fit(train_images, train_labels, epochs=5, batch_size=64, validation_split=0.1, verbose=1)

Step 8.4: Save training history and save the trained model for later evaluation.

Step 9 : Plot comparisons for validation accuracy and validation loss.

Step 10 : Evaluate each saved model on the test set. Record the final test accuracy for every optimizer and present results.

Step 11 : Analyze and conclude.

RESULT

The program was successfully executed and output was obtained

MNIST

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
train_images, test_images = train_images / 255.0, test_images / 255.0
train_images = train_images.reshape(train_images.shape[0], 28, 28, 1)
test_images = test_images.reshape(test_images.shape[0], 28, 28, 1)
train_labels = to_categorical(train_labels)
test_labels = to_categorical(test_labels)
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
    tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
    tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
history = model.fit(train_images, train_labels, epochs=5, batch_size=64, validation_split=0.1)
test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f"\nTest Accuracy: {test_acc * 100:.2f}%")
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
```

14 / 08 /2025

EXPERIMENT NO : 6

MNIST DIGIT CLASSIFICATION USING CNN ARCHITECTURE

AIM

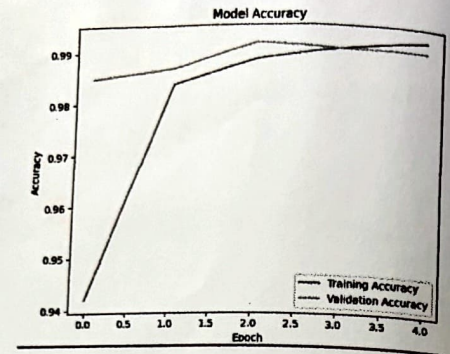
To implement a python program to implement digit classification using CNN architecture for MNIST dataset.

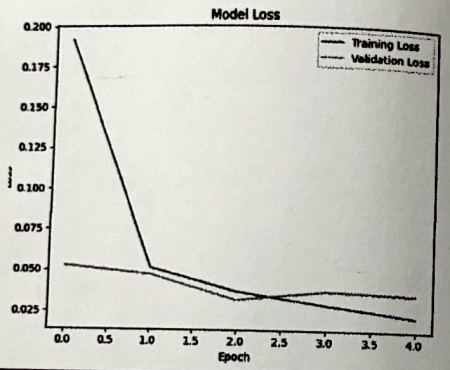
ALGORITHM

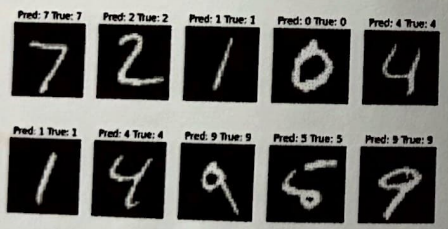
Step 1 : Import necessary libraries
Step 2 : Load the MNIST dataset.
Step 3 : Normalize images to scale pixel values to range [0, 1].
Step 4 : Reshape images into 4D tensors to match CNN input formats.
Step 5 : Convert labels into one-hot encoded vectors using to_categorical().
Step 6 : Build a sequential model with layers :
Step 6.1 : Conv2D layer with 64 filters, kernel size = (3x3), ReLU activation,
Step 6.2 : MaxPooling2D layer : pool size (2, 2)
Step 6.3 : Conv 2D layer with 64 filters of size 3x3 with ReLU activation
Step 6.4 : MaxPooling 2D layer with pool size 2x2
Step 6.5 : Conv2D layer with 64 filters of size 3x3 and ReLU activation
Step 6.6 : Output is flattened to a one-dimensional vector
Step 6.7 : Dense layer with 64 units and ReLU activation

```
    plt.ylabel('Loss')
    plt.legend()
    plt.tight_layout()
    plt.show()
predictions =
model.predict(test_images[:10])
def display_predictions(images,
predictions, labels):
    plt.figure(figsize=(10, 5))
    for i in range(10):
        plt.subplot(2, 5, i + 1)
        plt.imshow(images[i].reshape(28, 28),
cmap='gray')
        plt.title(f"Pred:
{predictions[i].argmax()} True:
{labels[i].argmax()}")
        plt.axis('off')
    plt.show()
display_predictions(test_images[:10],
predictions, test_labels[:10])
```

OUTPUT







/ /

Step 6.8: Dense layer with 10 units, and activation = 'softmax'

Step 7: Compile the model using Adam optimizer, categorical cross entropy loss and accuracy as the performance metric.

Step 8: Train the model for 5 epochs with a batch size of 64 with a 0.1 validation split.

Step 9: Evaluate the trained model on test data and record the accuracy.

Step 10: Plot graphs for training vs validation accuracy and training loss vs validation loss.

Step 11: Make predictions for the first 10 test images

Step 12: STOP

RESULT

Program was succesfully executed and output was obtained.

VGG-19

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import models, layers
from tensorflow.keras.applications import VGG19
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.preprocessing.image import smart_resize
import matplotlib.pyplot as plt
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)
x_train_resized = np.array([smart_resize(img, (48, 48)) for img in x_train])
x_test_resized = np.array([smart_resize(img, (48, 48)) for img in x_test])
x_train_rgb = np.repeat(x_train_resized, 3, axis=-1)
x_test_rgb = np.repeat(x_test_resized, 3, axis=-1)
x_train_rgb = x_train_rgb.astype("float32") / 255.0
x_test_rgb = x_test_rgb.astype("float32") / 255.0
vgg_model = VGG19(weights="imagenet", include_top=False, input_shape=(48, 48, 3))
for layer in vgg_model.layers:
    layer.trainable = False
model = models.Sequential([
    vgg_model,
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(10, activation="softmax")
])
model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
history = model.fit(
    x_train_rgb, y_train,
    epochs=10,
    batch_size=64,
    validation_data=(x_test_rgb, y_test),
    verbose=1
)
test_loss, test_acc = model.evaluate(x_test_rgb, y_test)
```

14/08/2025 14

EXPERIMENT NO: 7

DIGIT CLASSIFICATION ON MNIST USING VGG-19.

AIM

To implement a python program for digit classification using pre-trained networks like VGGnet-19, AlexNet, etc for MNIST dataset and analyze and visualize performance improvement.

ALGORITHM

Step 1 : Import required libraries.
Step 2 : Load the MNIST dataset and split it into training and testing sets.
Step 3 : Convert class labels into one-hot encoded vectors using to_categorical().
Step 4 : Expand dimensions of image to (28, 28, 1). Resize all images to (48,48) using smart_resize. Convert grayscale images to 3-channel RGB using np.repeat. Normalize pixel values to range [0, 1].
Step 5 : Import VGG19 model with imagenet weights Exclude the fully connected top layers. Set input shape to (48, 48, 3). Freeze all layers of VGG19.
Step 6 : Build a sequential model with the following structure
Step 6.1 : VGG-19 (pretrained, frozen)
Step 6.2 : Flatten layer

```
print(f"\nTest Accuracy: {round(test_acc * 100, 2)}%")

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)

plt.plot(history.history['accuracy'], label="Training Accuracy")

plt.plot(history.history['val_accuracy'], label="Validation Accuracy")

plt.title("Model Accuracy")

plt.xlabel("Epochs")

plt.ylabel("Accuracy")

plt.legend()

plt.subplot(1, 2, 2)

plt.plot(history.history['loss'], label="Training Loss")

plt.plot(history.history['val_loss'], label="Validation Loss")

plt.title("Model Loss")

plt.xlabel("Epochs")

plt.ylabel("Loss")

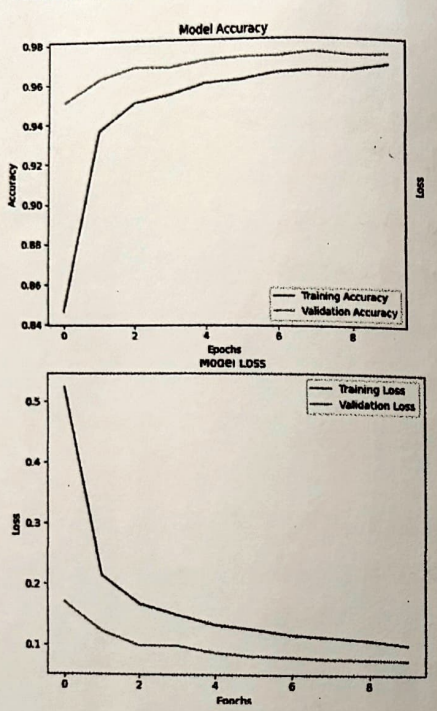
plt.legend()

plt.tight_layout()

plt.show()
```

OUTPUT

Test Accuracy: 97.41%



/ /

15

Step 6.3 : Dense layer with 128 units and ReLU activation.

Step 6.4 : Dropout layer with rate 0.5

Step 6.5 : Dense output layer with 10 units and softmax activation.

Step 7 : Compile the model using Adam optimizer, categorical cross entropy as loss function, and accuracy as evaluation metrics.

Step 8 : Train the model for 10 epochs with a batch size of 64 using the training set and validate using the test set.

Step 9 : evaluate the trained model on the test data and record the test accuracy.

Step 10 : Plot graphs for training vs validation accuracy and training vs validation loss. across epochs.

Step 11 : Analyze and conclude.

RESULT

Program was successfully executed and output was obtained.

RNN

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing import sequence
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
import matplotlib.pyplot as plt
max_features = 10000
maxlen = 200
batch_size = 32
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_features)
x_train = sequence.pad_sequences(x_train, maxlen=maxlen)
x_test = sequence.pad_sequences(x_test, maxlen=maxlen)
model = Sequential()
model.add(Embedding(max_features, 128))
model.add(SimpleRNN(128, activation="tanh"))
model.add(Dropout(0.5))
model.add(Dense(1, activation="sigmoid"))
model.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])
early_stopping = EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True)
history = model.fit(
    x_train, y_train,
    batch_size=batch_size,
    epochs=20,
    validation_split=0.2,
    callbacks=[early_stopping]
)
score, accuracy = model.evaluate(x_test, y_test, batch_size=batch_size)
print(f"Test score: {score:.4f}")
print(f"Test accuracy: {accuracy:.4f}")
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Train Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.subplot(1, 2, 2)
```


EXPERIMENT NO:8

REVIEW CLASSIFICATION USING RNN

AIM

To write and implement a python program for review classification on IMDB dataset using a simple Recurrent Neural Network.

ALGORITHM

Step 1 : Import necessary libraries
Step 2 : Load the IMDB dataset and split into training and testing sets.
Step 3 : Pad all input sequences to the same length (maxlen = 200) using sequences.pad_sequences
Step 4 : Define the sequential model
Step 4.1 : Add an embedding layer to represent words as dense vectors of dimension 128.
Step 4.2 : Add a simple RNN layer with 128 units and tanh activation to process the sequence.
Step 4.3 : Add a Dropout (0.5) layer to reduce overfitting.
Step 4.4 : Add a final Dense (1, sigmoid) output layer for binary classification.
Step 5 : Compile the model with optimizer = 'Adam', loss = 'binary_crossentropy' and metric = 'Accuracy'
Step 6 : Use early stopping with monitor = "val_loss",

```
plt.plot(history.history["loss"],
label="Train Loss")
plt.plot(history.history["val_loss"],
label="Validation Loss")
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.tight_layout()
plt.show()
word_index = imdb.get_word_index()
reverse_word_index = {value: key for key,
value in word_index.items()}
def decode_review(review):
    return "
".join([reverse_word_index.get(i - 3, "?")
for i in review])
def preprocess_reviews(reviews):
    encoded_reviews = []
    for review in reviews:
        encoded_review =
[word_index.get(word.lower(), 0) + 3 for
word in review.split()]
        encoded_reviews.append(encoded_r
eview)
    return
sequence.pad_sequences(encoded_revie
ws, maxlen=maxlen)
sample_reviews = [
  "This movie was fantastic! I loved it.",
  "I didn't like this film at all. It was boring
and too long.",
  "An average film, nothing special."
]
encoded_sample_reviews =
preprocess_reviews(sample_reviews)
predictions =
model.predict(encoded_sample_reviews)
predicted_classes = (predictions >
0.5).astype("int32")
for review, prediction in
zip(sample_reviews, predicted_classes):
  sentiment = "Positive" if prediction[0]
== 1 else "Negative"
  print(f"Review: {review}\nPredicted
Sentiment: {sentiment}\n")
```


patience = 8, and restore_best_weights = True to stop training if validation loss doesnot improve.

Step 7 : Train the model for upto 80 epochs and batch_size = 32, validation_split = 0.2 and save training history for plotting.

Step 8 : Evaluate on the test dataset and record loss and test accuracy.

Step 9 : Plot training vs validation accuracy for epochs and training vs validation loss across epochs.

Step 10 : Decode and preprocess new reviews.

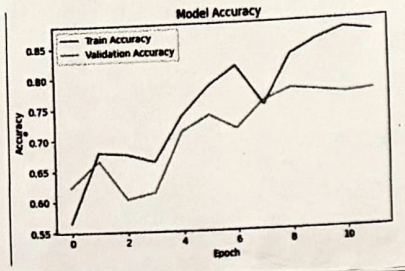
Step 11 : Predict sentiment of sample reviews using trained model.

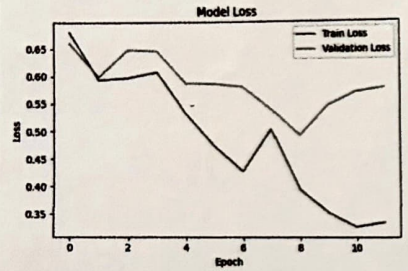
Step 12 : Analyze and conclude.

OUTPUT

Test score: 0.4864

Test accuracy: 0.7801





Review: This movie was fantastic! I loved it.

Predicted Sentiment: Positive

Review: I didn't like this film at all. It was boring and too long.

Predicted Sentiment: Negative

Review: An average film, nothing special.

Predicted Sentiment: Negative

/ /

18

RESULT

Program was successfully executed and output was obtained.

LSTM and GRU

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing import sequence
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Embedding, SimpleRNN, LSTM, GRU
max_features = 20000
maxlen = 100
batch_size = 32
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_features)
x_train = sequence.pad_sequences(x_train, maxlen=maxlen)
x_test = sequence.pad_sequences(x_test, maxlen=maxlen)
def create_model(rnn_type, units=128):
    model = Sequential()
    model.add(Embedding(max_features, 128))
    if rnn_type == "SimpleRNN":
        model.add(SimpleRNN(units))
    elif rnn_type == "LSTM":
        model.add(LSTM(units))
    elif rnn_type == "GRU":
        model.add(GRU(units))
    model.add(Dense(1, activation="sigmoid"))
    model.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])
    return model
rnn_types = ["SimpleRNN", "LSTM", "GRU"]
histories = {}
for rnn_type in rnn_types:
    print(f"Training {rnn_type} model...")
    model = create_model(rnn_type)
    history = model.fit(
        x_train, y_train,
        batch_size=batch_size,
        epochs=5,
        validation_data=(x_test, y_test),
        verbose=1
    )
    histories[rnn_type] = history
    test_loss, test_acc = model.evaluate(x_test, y_test)
    print(f"{rnn_type} Test Accuracy: {test_acc:.4f}")
def plot_history(histories, metric="accuracy"):
    plt.figure(figsize=(12, 6))
    for rnn_type, history in histories.items():
```

14 / 08 /2025 | 19

EXPERIMENT NO : 9

SENTIMENT ANALYSIS USING LSTM AND GRU

AIM

To write a python program to analyze and visualize performance change while using LSTM and GRU instead of simple RNN.

ALGORITHM

Step 1 : Import libraries and set seeds
Step 2 : Set hyperparameters
Step 3 : Load IMDB dataset and pad sequences to fixed length.
Step 4 : Create fixed training, validation and test split.
Step 5 : Define a function to create model by building a sequential network that includes an Embedding layer(128) followed by a recurrent layer (SimpleRNN, LSTM or GRU depending on the chosen type), and finally a dense output layer with sigmoid activation function for binary sentiment classification, where max features or units = 128.
Step 6 : Compile the model use loss = 'binary_crossentropy', optimizer = 'adam' and mebics = ['Accuracy']
Step 7 : For each RNN type (SimpleRNN, LSTM, GRU)
Step 7.1 : Train for 5 epochs with batch size = 32.
Step 7.2 : Use validation data to track validation performance.
Step 7.3 : Store training history for comparison.

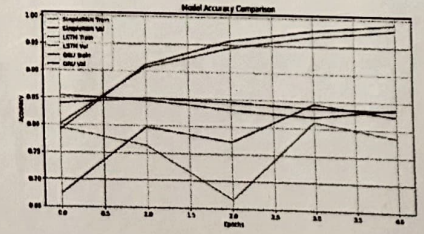
```
    plt.plot(history.history[metric],
label=f"{rnn_type} Train")
    plt.plot(history.history["val_" +
metric], label=f"{rnn_type} Val")
  plt.title(f"Model {metric.capitalize()}
Comparison")
  plt.xlabel("Epochs")
  plt.ylabel(metric.capitalize())
  plt.legend()
  plt.grid(True)
  plt.show()
plot_history(histories, "accuracy")
plot_history(histories, "loss")
```

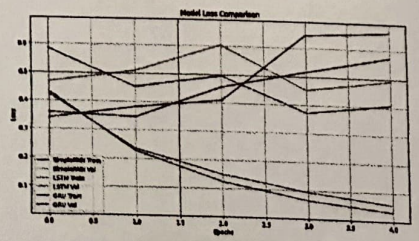
OUTPUT

SimpleRNN Test Accuracy: 0.7856

LSTM Test Accuracy: 0.8346

GRU Test Accuracy: 0.8381





/ /

Step 8 : Evaluate each model on test data to record test accuracy and loss.

Step 9 : Plot training vs validation accuracy curves and training vs validation loss curves.

Step 10 : Compare model performance

Step 11 : Analyze result and conclude.

RESULT

Program was successfully executed and output was obtained.

PROGRAM

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
df = yf.download("^NSEI", start="2010-01-01", end="2024-01-01")
df = df[['Close']]
df.dropna(inplace=True)
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(df)
def create_sequences(data, seq_length):
    X, y = [], []
    for i in range(seq_length, len(data)):
        X.append(data[i - seq_length:i, 0])
        y.append(data[i, 0])
    return np.array(X), np.array(y)
seq_length = 60  # 60 days look-back
X, y = create_sequences(scaled_data, seq_length)
X = np.reshape(X, (X.shape[0], X.shape[1], 1))
model = Sequential()
model.add(LSTM(units=50, return_sequences=True, input_shape=(X.shape[1], 1)))
model.add(Dropout(0.2))
model.add(LSTM(units=50, return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(units=1))
```


EXPERIMENT NO: 10

TIME SERIES FORECASTING PREDICTION

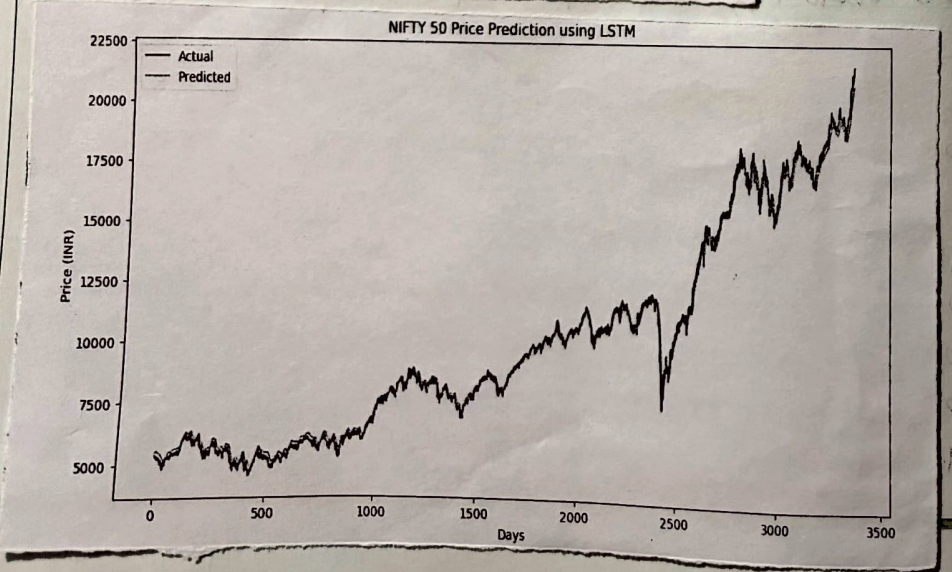
AIM

To implement time series forecasting prediction for NIFTY-50 dataset.

ALGORITHM

Step 1 : START
Step 2 : Import the necessary libraries
Step 3 : Download the NIFTY-50 Index from YahooFinance and keep only the close price colourmn.
Step 4 : Normalize the data by applying Min-Max scaling, converting all prices to a range between 0 and 1.
Step 5 : Create input and output sequences where each input consists of the previous 60 days prices and the output is the next days price.
Step 6 : Reshape the input data into 3D format suitable for LSTM
Step 7 : Build the LSTM Model :
Step 7.1: Add an LSTM layer with 50 units and return sequences
Step 7.2: Add a Dropout layer to reduce overfitting
Step 7.3: Add another LSTM layer with 50 units.
Step 7.4: Add another Dropout layer.

```
model.compile(optimizer='adam', loss='mean_squared_error')
model.summary()
history = model.fit(X, y, epochs=20, batch_size=32)
last_sequence = scaled_data[-seq_length:]
last_sequence = np.reshape(last_sequence, (1, seq_length, 1))
predicted_price = model.predict(last_sequence)
predicted_price = scaler.inverse_transform(predicted_price)
print(f"Predicted Next Day Closing Price: ₹{predicted_price[0][0]:.2f}")
train_predict = model.predict(X)
train_predict = scaler.inverse_transform(train_predict)
y_actual = scaler.inverse_transform(y.reshape(-1, 1))
plt.figure(figsize=(12, 6))
plt.plot(y_actual, label='Actual')
plt.plot(train_predict, label='Predicted')
plt.title('NIFTY 50 Price Prediction using LSTM')
plt.xlabel('Days')
plt.ylabel('Price (INR)')
plt.legend()
plt.show()
```



/ /

Step 7.5: Add a Dense layer with one output unit.
Step 8 : Compile the model using the Adam optimizer and Mean Squared Error (MSE) as the loss function.
Step 9 : Train the model on the prepared data for 20 epochs with a batch size of 32.
Step 10 : Use the trained model to predict the next day's closing price on the last 60 days of input data.
Step 11 : Inverse transform the predicted value to obtain the actual price in Rupees.
Step 12 : Plot the graph the visual comparison by showing the actual vs. predicted stock prices.
Step 13 : Display the predicted next day closing price.
Step 14 : STOP

RESULT

The program was successfully executed and output is obtained.

27.10

PROGRAM

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.models import Model

from tensorflow.keras.layers import Input, LSTM, Dense, Embedding

from tensorflow.keras.preprocessing.text import Tokenizer

from tensorflow.keras.preprocessing.sequence import pad_sequences

from sklearn.model_selection import train_test_split

from tensorflow.keras.utils import to_categorical

import matplotlib.pyplot as plt

file_path ='/content/hindi_english_parallel.csv'

english_sentences, hindi_sentences = [], []

with open(file_path, 'r', encoding='latin-1') as f:

  for line in f:

    parts = line.split("\t")

    if len(parts) == 3:

      eng, hin, _ = parts

      english_sentences.append(eng.strip())

      hindi_sentences.append(hin.strip())

english_tokenizer = Tokenizer(char_level=True)

hindi_tokenizer = Tokenizer(char_level=True)

english_tokenizer.fit_on_texts(english_sentences)

hindi_tokenizer.fit_on_texts(hindi_sentences)

eng_sequences = english_tokenizer.texts_to_sequences(english_sentences)

hin_sequences = hindi_tokenizer.texts_to_sequences(hindi_sentences)

max_eng_len = max(len(seq) for seq in eng_sequences)

max_hin_len = max(len(seq) for seq in hin_sequences)

eng_padded = pad_sequences(eng_sequences, maxlen=max_eng_len, padding='post')

hin_padded = pad_sequences(hin_sequences, maxlen=max_hin_len, padding='post')

num_decoder_tokens = len(hindi_tokenizer.word_index) + 1

hin_padded_shifted = np.array([to_categorical(seq, num_classes=num_decoder_tokens) for seq in

hin_padded[:, 1:]])

eng_train, eng_test, hin_train, hin_test = train_test_split(eng_padded, hin_padded,

test_size=0.2,random_state=42)

hin_train_shifted, hin_test_shifted = train_test_split(hin_padded_shifted,

test_size=0.2,random_state=42)

latent_dim=256

encoder_inputs = Input(shape=(max_eng_len,))
```

08 / 09 / 25

EXPERIMENT NO: 11

SHALLOW AUTOENCODER AND DECODER NETWORK FOR MACHINE TRANSLATION

AIM

To implement a shallow autoencoder and decoder network for machine translation

ALGIORITHM

Step 1 : START
Step 2 : Load Kaggle English to Hindi neural translation dataset
Step 3 : Load English - Hindi parallel sentences from the dataset.
Step 4 : Add start and end tokens to Hindi sentences for the decoder.
Step 5 : Tokenize English and Hindi sentences at the character level.
Step 6 : Convert sentences to integer sequences and pad them for decoder output.
Step 7 : Shift Hindi sequences by one time step and one-hot encode them for decoder output.
Step 8 : Split the data into training and testing sets.
Step 9 : Define an encoder LSTM with embedding and capture its states.
Step 10 : Define a decoder LSTM with embedding, initialize it with

```
encoder_embedding =
Embedding(input_dim=len(english_tokenizer.
word_index) + 1,
output_dim=latent_dim)(encoder_inputs)

encoder_outputs, state_h, state_c =
LSTM(latent_dim,
return_state=True)(encoder_embedding)

encoder_states = [state_h, state_c]

decoder_inputs = Input(shape=(max_hin_len
- 1,))

decoder_embedding =
Embedding(input_dim=num_decoder_tokens
,
output_dim=latent_dim)(decoder_inputs)

decoder_lstm = LSTM(latent_dim,
return_sequences=True, return_state=True)

decoder_outputs, _, _ =
decoder_lstm(decoder_embedding,
initial_state=encoder_states)

decoder_dense =
Dense(num_decoder_tokens,
activation='softmax')

decoder_outputs =
decoder_dense(decoder_outputs)

model = Model([encoder_inputs,
decoder_inputs], decoder_outputs)

model.compile(optimizer='adam',
loss='categorical_crossentropy',
metrics=["accuracy"])

batch_size = 64

epochs = 20

history = model.fit([eng_train, hin_train[:, :-
1]], hin_train_shifted,batch_size=batch_size,
epochs=epochs, validation_split=0.2)

model.save("eng_to_hin_translation_model.
h5")

loss, accuracy = model.evaluate([eng_test,
hin_test[:, :-1]], hin_test_shifted)

print(f"Test Accuracy: {accuracy * 100:.2}%")

plt.plot(history.history['accuracy'],
label='Training Accuracy')

plt.plot(history.history['val_accuracy'],
label='Validation Accuracy')

plt.xlabel('Epoch')

plt.ylabel('Accuracy')

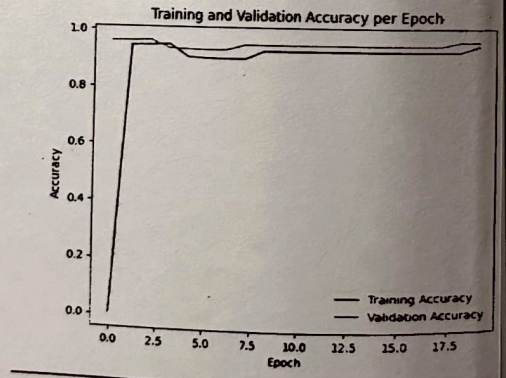
plt.legend()

plt.title("Training and Validation Accuracy per
Epoch")

plt.show()
```

OUTPUT

OUTPUT



encoder states, and apply a Dense layer with softmax activation.

Step 11 : Compile the model using categorical cross-entropy loss and the Adam optimizer.

Step 12 : Train the model on training data with teacher forcing and validate on a portion of it.

Step 13: Evaluate the model on training data with teacher forcing and validate on a portion of it.

Step 14: Save the trained model for future inference and plot accuracy per epoch.

Step 15 : STOP.

RESULT

The program was successfully executed and output was obtained.

27.10.

PROGRAM

```
!pip install transformers torch
from transformers import pipeline
def analyze_sentiment(text):
    sentiment_pipeline = pipeline("sentiment-analysis", model="distilbert-base-uncased-finetuned-sst-2-english")
    result = sentiment_pipeline(text)[0]
    return result
# Example 1: Positive sentiment
positive_sentence = "I absolutely love this new phone, it's incredible!"
positive_result = analyze_sentiment(positive_sentence)
print(f"Sentence: '{positive_sentence}'")
print(f"Sentiment: {positive_result['label']} (Score: {positive_result['score']:.4f})")
print("-" * 20)
#Example 2: Negative sentiment
negative_sentence = "This movie was a complete waste of time, I'm so disappointed."
negative_result = analyze_sentiment(negative_sentence)
print(f"Sentence: '{negative_sentence}'")
print(f"Sentiment: {negative_result['label']} (Score: {negative_result['score']:.4f})")
print("-" * 20)
# Example 3: Neutral/Mixed sentiment
neutral_sentence = "The food was okay, but the service was terrible."
neutral_result = analyze_sentiment(neutral_sentence)
print(f"Sentence: '{neutral_sentence}'")
print(f"Sentiment: {neutral_result['label']} (Score: {neutral_result['score']:.4f})")
```

15/09/25

25

EXPERIMENT NO :12

SENTIMENT ANALYSIS USING PRE-TRAINED BERT

AIM

To implement sentiment analysis using a pre-trained BERT model to classify text as positive or negative.

ALGORITHM

Step 1 : START

Step 2 : Install the required libraries : transformers and torch for using pre-trained NLP models

Step 3 : Import the pipeline function from the transformers library

Step 4 : Define a function named as 'analyze_sentiment(text)' to analyze sentiment of the given text.

Step 5 : Inside the function, create a sentiment-analysis pipeline using the pre-trained model.

Step 6 : Pass the input text to the pipeline to predict the sentiment and obtain its confidence score

Step 7 : Return the sentiment label and score as output.

Step 8 : Provide the three sample input sentences : a positive sentence, a negative sentence and a neural or mixed sentence.

Step 9 : Call the analyze_sentiment() function for each example sentence.

Step 10 : Display the original sentence, predicted sentiment label

OUTPUT

Sentence: 'I absolutely love this new phone, it's incredible!'

Sentiment: POSITIVE (Score: 0.9999)

Sentence: 'This movie was a complete waste of time, I'm so disappointed.'

Sentiment: NEGATIVE (Score: 0.9998)

Sentence: 'The food was okay, but the service was terrible.'

Sentiment: NEGATIVE (Score: 0.9862)

and confidence score

Step 11 : STOP

RESULT

Program was successfully executed and output was obtained.

27.10